SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I Divyashree Koteeswaran / 1921260041 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Course Faculty

Annexure A

DESIGN TASK

Q4. Critically evaluate the effectiveness of Parsing Expression Grammars (PEG) as a modern parsing technique by comparing them with traditional LL and LR parsers in terms of ambiguity handling, grammar expressiveness, implementation complexity, performance, and real-world applicability.

Your answer should:

1. **Analyze** how PEG parsers resolve ambiguity differently from CFG-based parsers
2. **Compare** PEG with LL and LR parsing in terms of:
   - Grammar expressiveness
   - Ease of implementation
   - Performance and backtracking
3. **Examine** real-world adoption (e.g., Python's PEG parser) and justify why PEG was preferred
4. **Critically evaluate** limitations of PEG (e.g., lack of left recursion support, ordered choice behavior)
5. **Conclude** whether PEG is suitable for all programming languages or only specific cases, with justification

Answer:

1. Introduction

A Parsing Expression Grammar (PEG) is a formal grammar used to describe the syntax of programming languages. It uses ordered choice to select between alternative parsing rules.

Unlike Context-Free Grammars (CFGs), PEG does not treat alternatives as unordered choices.

2. Ambiguity Handling

In PEG, / represents ordered choice.

Example:

```
A <- "ab" / "a"
```

The parser tries "ab" first. If it succeeds, that alternative is selected. If it fails, the parser tries "a".

Therefore, the order of alternatives affects the parsing result.

3. Comparison of PEG, LL and LR

Feature	PEG	LL Parser	LR Parser
Parsing method	Top-down	Top-down	Bottom-up
Ambiguity	Uses ordered choice	Requires suitable grammar	Requires suitable grammar
Backtracking	May occur	LL(1) avoids it	Standard LR avoids it
Left recursion	Requires special support or rewriting	Must be eliminated	Supported
Implementation	Convenient with suitable tools	Simple for LL(1)	More complex

Memory usage	Can be high with memoization	Generally low	Depends on table size

4. Applications

Python adopted a PEG-based parser in Python 3.9, replacing its earlier LL(1)-based parser.

PEG provides flexible grammar descriptions and makes certain language constructs easier to express.

5. Limitations

- Ordered choice makes alternative order important.
- Traditional PEG parsers have difficulties with left recursion.
- Packrat parsing may consume considerable memory.
- Error reporting can require additional work.

6. Conclusion

PEG is useful for programming languages because it provides flexible grammar descriptions and predictable alternative selection. However, LL and LR parsers may be more suitable for certain applications.

Q10. Design a Hybrid Parsing Strategy

Task:

Design a hybrid parsing system that combines LL(1) and LR parsing techniques to efficiently parse a programming language with both simple and complex constructs.

Conditions:

Identify which parts use LL(1) vs LR parsing

Justify the hybrid design choice

Define grammar transformations required

Illustrate parsing workflow with diagram

Demonstrate parsing of a sample input

Evaluate advantages over standalone methods

Test Case:

Input: id + id * id

Answer:

1. Introduction

A hybrid parser combines two parsing techniques to handle different parts of a programming language.

In this design, LL(1) parses simple statements, while LR parses complex expressions.

2. Grammar

Consider the input: a = b + c * d

Statement grammar:

Stmt $\rightarrow$ id = E

Expression grammar:

E $\rightarrow$ E + T | T
T $\rightarrow$ T * F | F

$F \rightarrow id$

The LL(1) parser recognizes the assignment statement, and the LR parser handles the expression.

3. Working Procedure

1. The lexical analyzer converts source code into tokens.
2. The LL(1) parser identifies the assignment statement.
3. The expression tokens are passed to the LR parser.
4. The LR parser handles multiplication and addition according to precedence.
5. The syntax tree is constructed.
6. The compiler can generate intermediate code.

4. Parsing Example

Input: id + id * id

The LR parser follows these reductions:

$F \rightarrow id$

$T \rightarrow F$

$E \rightarrow T$

After recognizing the addition operator, it parses the next term:

$T \rightarrow T * F$

Finally:

$E \rightarrow E + T$

The resulting grouping is: id + (id * id)

5. Advantages

- LL(1) simplifies statement parsing.
- LR handles left-recursive expression grammars.
- The design separates different grammar components.
- Each parser can be tested independently.

6. Limitations

- Communication between parsers must be carefully designed.
- Parser boundaries must be clearly defined.
- Error recovery can become complicated.
- Maintaining two parsing systems increases implementation effort.

7. Conclusion

The hybrid approach combines the simplicity of LL(1) with the parsing capabilities of LR. It is useful when a programming language contains simple statements and complex expressions.

Q14. Create a PEG-Based Parsing System

Task:

Design a parsing system using Parsing Expression Grammars (PEG) for a programming language and evaluate its performance against CFG-based parsers.

Conditions:

Convert CFG to PEG format

Handle ordered choice and backtracking

Implement parsing logic

Compare ambiguity handling

Analyze performance and limitations

Demonstrate parsing process

Test Case:
Input: id + id * id

Answer:

1. Introduction

A Parsing Expression Grammar (PEG) is a top-down parsing method that uses ordered choice and repetition.

The goal is to parse arithmetic expressions while maintaining operator precedence.

2. PEG Grammar

```
E <- T ("+" T)*
T <- F ("*" F)*
F <- id
```

Here:

- E handles addition.
- T handles multiplication.
- F handles identifiers.
- * means zero or more repetitions.

3. Parsing Example

Input: id + id * id

The parser performs these steps:

1. Recognizes the first id.
2. Recognizes +.
3. Recognizes the second id.
4. Recognizes *.
5. Recognizes the third id.
6. Completes the multiplication and addition expressions.
7. Accepts the input if the entire input has been consumed.

4. Syntax Tree

```
    +
   / \
 id   *
     / \
    id  id
```

The tree shows that multiplication has higher precedence than addition.

5. PEG vs CFG

Feature	PEG	CFG
Alternatives	Ordered	Unordered
Ambiguity	Ordered choice selects a result	May allow multiple parse trees
Left recursion	Requires special support or rewriting in conventional PEGs	Can represent left recursion
Parsing strategy	Usually top-down	Depends on parser

Performance	Packrat parsing can provide linear-time recognition with additional memory	Depends on parsing method

6. Advantages and Limitations

Advantages:

- Clear grammar notation.
- Predictable alternative selection.
- Convenient top-down parsing.

Limitations:

- Alternative order affects the result.
- Packrat parsing may use significant memory.
- Traditional PEG parsers need special handling of left recursion.

7. Conclusion

The PEG-based parser accepts id + id * id and preserves multiplication precedence over addition.

PEG is useful for language parsing, but its memory requirements and left-recursion limitations must be considered.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/ Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation

CO2 - AT Design Task.

Divyasree Kotteeswaran.
Reg. No: 192126004
Course Code: CSA - 1409.

Q.1. Predictive parser design (LL(1))

Given:

$E \rightarrow TE'$

$T \rightarrow FT'$

$F \rightarrow (E) \mid id$

Input: id + id * id.

Step-1 First set.

Non-terminal	First.
E	{id, (}
E'	{+, ε}
T	{id, (}
T'	{*, ε}
F	{id, (}

Step-2 follow set.

Non-terminal	Follow
E	{), $}
E'	{), $}
T	{+,), $}
T'	{+,), $}
F	{*, +,), $}

Step-3 LL(1) Parsing table.

Non-terminal	id	+	*	(	)	$
E	TE'	-	-	TE'	-	-
E'	-	+TE'	-	-	ε	ε
T	FT'	-	-	FT'	-	-
T'	-	ε	*FT'	-	ε	ε
F	id	-	-	(E)	-	-

Scanned with OKEN Scanner

Step-4 Parsing Trace.

Stack.	Input	Action.
E$	id + id * id $	E → TE'
TE'$	id + id * id $	T → FT'
FT'E'$	id + id * id $	F → id
idT'E'$	id + id * id $	T' → ε match id.
T'E'$	+ id * id $	T' → ε.
E'$	+ id * id $	E' → +TE'
+TE'$	+ id * id $	match +
TE'$	id * id $	T → FT'
FT'E'$	id * id $	F → id
idT'E'$	id * id $	match id.
T'E'$	$	T' → ε.
E'$	$	E' → ε.
$	$	Accept.

Q.2) Shift-Reduce parsing.

Given: E → E+E | E*E | id.

Stack.	Input	Action.
$	id + id * id $	Shift id.
$id	+ id * id $	Reduce E → id
$E	+ id * id $	Shift +
$E+	id * id $	Shift id.
$E+id	* id $	Reduce E → id.
$E+E	* id $	Reduce E → E+E
$E	* id $	Shift *
$E*	id $	Shift id.
$E*id	$	Reduce E → id.
$E*E	$	Reduce E → E*E
$E	$	Accept.

Scanned with OKEN Scanner